

Resort ATV Voice Assistant

A hybrid local/cloud, trilingual voice assistant built into an electric ATV

Project overview and technology guide — updated 2026-08-10

1. What This Project Is

This is a voice assistant meant to be built directly into an electric all-terrain vehicle (ATV) at a resort — not a phone app. Mics and speakers are physically mounted on the vehicle, so any guest can talk to it directly, the way you'd talk to a car's built-in assistant or a smart speaker.

Right now, everything runs on a laptop as a prototype, standing in for the vehicle's future onboard computer. A rider says a wake phrase ("Hey EV"), asks a question about the vehicle (battery level, speed, tire pressure, motor temperature), and the assistant answers out loud — in English, Hindi, or Tamil, whichever language was spoken, automatically detected with no manual switch.

The long-term vision (not built yet) is for this assistant to become the primary way riders interact with a fully autonomous vehicle — booking rides, getting guided tours, returning to a charging station on its own, and warning if the vehicle leaves resort property. None of that exists yet; what's built today is the voice interaction layer and the vehicle-status Q&A.

Updated 2026-08-10: this project was originally built to run 100% offline with zero cloud calls. That's still true for the vehicle-critical parts (telemetry answers and small talk), but as of today, questions that aren't about the vehicle (weather, general knowledge, casual chat) are answered by Google's Gemini AI instead of a local model — see Section 2 and 4 for why.

2. How It Works — The Pipeline

Every voice interaction flows through the same stages, in order:

1 Wake Word Detection — listens continuously for "Hey EV"

2 Speech-to-Text (STT) — converts what you said into written text, and figures out which of the 3 languages you spoke

3 The Router — a small local AI model decides: "is this a question about the vehicle's status, or something else?"

4a Vehicle Status Lookup (local) — if it's a vehicle question, the exact number is fetched from a data cache (never guessed or invented) — fully offline, instant

4b Gemini Chat (cloud) — if it's anything else (weather, general knowledge, casual conversation), the question is sent to Google's Gemini AI, which answers in the same language you spoke

5 Text-to-Speech (TTS) — converts the answer back into spoken audio through the speaker

Why vehicle questions never go through step 4b, and why that matters: Early versions let an AI model freely generate an answer to anything, including vehicle stats. In testing, it sometimes *invented* things — a fake speed reading, a made-up resort activity, even answering an English question in Tamil by mistake. For anything involving a real vehicle statistic, this project deliberately does **not** let an AI "guess" — the actual number always comes from a direct, deterministic lookup (step 4a), never generated text. This is the single most important design decision in the whole project: *never let the AI state a fact it wasn't given directly*.

Why Gemini for the "everything else" case (step 4b): Two different local AI models were tried for open-ended questions and both kept fabricating things anyway, despite being explicitly told not to — the same underlying reliability problem the callout above describes, just for non-vehicle questions instead of vehicle ones. Gemini is a much larger, more capable model that was directly verified to handle this honestly — asked live "what activities does the resort have," it correctly said it didn't know instead of inventing one. The tradeoff: this one step needs an internet connection and takes a few seconds longer than the fully local path.

3. Tech Stack — What Each Piece Is and Does

3.1 Language & Foundations

Technology	What It Is	Its Role Here
Python	A widely-used, beginner-friendly programming language. Almost all AI/ML tooling is built for it.	The language the entire assistant is written in.
PyTorch (torch)	The most popular software framework for building and running AI/deep-learning models.	Powers the text-to-speech models for Hindi/Tamil, and was used to train the custom wake word.
NVIDIA CUDA / GPU	CUDA is NVIDIA's technology that lets AI software run on a graphics card (GPU) instead of just the processor (CPU). GPUs do the specific kind of math AI models need far faster.	The laptop has an NVIDIA RTX 3050 GPU. Using it sped up speech recognition by roughly 12x compared to running on the CPU alone.

3.2 Stage 1 — Wake Word Detection

Technology	What It Is	Its Role Here
openWakeWord	An open-source software library specialized in one job: constantly listening to a microphone stream and detecting one specific trigger phrase, efficiently, with very low resource use.	Runs continuously in the background. When it hears "Hey EV" spoken, it triggers the rest of the pipeline to wake up and start listening for a real question.

livekit-wakeword	A separate tool used only <i>once</i> , offline, to <i>create</i> a custom wake-word model — not used while the assistant is actually running.	Used to train the custom "Hey EV" detection model from scratch, using computer-generated speech samples instead of real human recordings. Live-tested with a real human voice and confirmed working reliably.
------------------	--	---

3.3 Stage 2 — Speech-to-Text

Technology	What It Is	Its Role Here
Whisper (faster-whisper, model size large-v3-turbo)	An AI speech-recognition model originally built by OpenAI. It listens to audio and writes down what was said. This project uses faster-whisper, an optimized version that runs the same model faster and lighter.	Transcribes the rider's spoken question into text, and separately identifies which of the three supported languages (English, Hindi, Tamil) was spoken. Runs entirely on the GPU, locally — no internet needed for this step.

3.4 Stage 3 — The Router (Local Decision-Making AI)

Technology	What It Is	Its Role Here
Qwen2.5 (1.5B)	A small, open-source "large language model" (LLM) — the same broad category of AI as ChatGPT or Gemini, but small enough (1.5 billion parameters) to run entirely on a laptop with no internet connection.	Reads the transcribed question and classifies it: is this asking about a vehicle statistic (and if so, which one), or is it something else that should go to Gemini instead? Runs entirely locally.
llama.cpp / llama-cpp-python	A software engine that runs large language models efficiently on ordinary computer hardware, without needing specialized cloud servers.	The engine actually running the Qwen2.5 router model on this laptop's CPU.
GBNF Grammar	A formal rulebook that can force an AI model's output into an exact, predictable shape — in this case, only ever one of two specific JSON formats — instead of letting it write free-form text.	This is the safety mechanism that stops the router from ever inventing an answer: it is physically incapable of outputting anything except "fetch this specific field" or "this is something else." It can never output a made-up number.

3.5 Stage 4a — Vehicle Data (Local, Deterministic)

Technology	What It Is	Its Role Here
------------	------------	---------------

CAN bus /
python-can

CAN (Controller Area Network) is the real, industry-standard electrical protocol that virtually every modern car and vehicle uses internally for its computers to talk to each other and report sensor data. `python-can` is a Python toolkit for reading and writing that protocol.

No real ATV hardware exists yet, so this project simulates a CAN bus in software (a "virtual" bus) with a fake program standing in for the vehicle's real onboard computer, broadcasting battery/speed/temperature/tire-pressure readings the same way a real vehicle would. Fully local — this step never touches the network.

3.6 Stage 4b — General Chat (Cloud)

Technology	What It Is	Its Role Here
Google Gemini (<code>gemini-flash-latest</code>)	A large, cloud-hosted AI model from Google — the same broad category as Qwen2.5 above, but much bigger, more capable, and running on Google's servers rather than this laptop.	Answers anything that isn't a vehicle question — weather, general knowledge, casual conversation — in whichever language the rider spoke. This is the only part of the live app that ever makes a network call; if there's no internet, it fails gracefully with an honest spoken message instead of crashing.

3.7 Stage 5 — Text-to-Speech

Technology	What It Is	Its Role Here
Piper TTS	An open-source, offline text-to-speech engine — converts written text into natural-sounding spoken audio.	Speaks all English responses, using a British voice ("en_GB-cori-high"). Fully local.
MMS-TTS (Meta)	A text-to-speech model family from Meta (Facebook's parent company) specifically built to cover many of the world's languages, including ones most TTS engines skip.	Speaks all Hindi and Tamil responses — Piper and other evaluated alternatives (Kokoro-82M, MeloTTS, Indic Parler-TTS) don't cover Tamil at all, or were too slow. Fully local.
<code>sounddevice</code>	A Python library for capturing microphone audio and playing sound through speakers.	Handles all actual audio input/output — recording the rider's voice and playing the spoken reply.

3.8 Supporting Tools (not part of the live pipeline, but essential to building it)

Technology	What It Is	Its Role Here
------------	------------	---------------

pytest	A framework for writing and running automated tests — small programs that check other code actually works correctly.	242+ automated tests exist, checking everything from number-formatting to routing decisions to the Gemini fallback logic, so future changes can be verified quickly instead of by hand every time.
ruff & mypy	Two "code quality" tools. <code>ruff</code> checks for style issues and common mistakes; <code>mypy</code> checks that the code's data types are used consistently, catching a class of bug before the program ever runs.	Run against the whole codebase; both found and helped fix real (if minor) bugs.
Git & GitHub	Git tracks every change ever made to the code as a history you can look back through or undo. GitHub is a website that hosts a backup copy of that history online.	Every change to this project is committed to Git; the full history is backed up at github.com/spurgen026/ATV-VOICE-PROJECT .

3.9 Parked / Not Currently Used (built, then intentionally set aside)

Technology	What It Was	Current Status
Local chat models (Qwen2.5-3B, Tamil-Llama-7B)	Two local AI models that used to handle non-vehicle questions before Gemini replaced them today - one general-purpose, one specifically trained for better Tamil.	Removed from the live app after repeatedly fabricating things (invented resort activities, a fake speed reading) despite being told not to. Kept in the codebase, unused, in case a safer local approach is worth revisiting later.
V2's Gemini document search (RAG)	An earlier version searched actual uploaded documents (like resort policy PDFs) and had Gemini answer using only that retrieved text.	Replaced by the current architecture. Would still be useful if real resort documents (hours, policies) existed to index - none do yet. Kept in the codebase, unused.

4. Current Status

Component	Status	Detail
Wake word ("Hey EV")	Working	Custom-trained, live-tested by real voice, confirmed reliable.
Speech-to-text (English/Hindi/Tamil)	Working	GPU-accelerated, auto-detects language per question.
Vehicle telemetry Q&A	Working	Battery, motor temperature, speed, tire pressure - all answered from real (simulated) data, never guessed, fully offline, instant.
General/open-ended questions	Working (cloud)	Handled by Gemini as of 2026-08-10 - verified honest ("I don't know") on questions it genuinely can't answer, rather than inventing an answer.
Text-to-speech (3 languages)	Working	Distinct voices for English, Hindi, and Tamil.
Vehicle-critical functions offline	Confirmed	Telemetry and small talk verified working with the network physically disconnected.
Tamil routing accuracy	Partial	Improved today (added colloquial Tamil/Hindi vocabulary like "charge," "air," "hot"), but a handful of heavily garbled Tamil phrasings are still known to be misunderstood.
Real vehicle hardware	Not started	Everything currently runs on a laptop. Target hardware (Raspberry Pi 5 and/or NVIDIA Jetson) has not been tested - a real risk, since the laptop's GPU speed boost may not be available on the final hardware.
Automated CI testing	Not active	Attempted, hit a real Linux-specific dependency issue (missing system audio library), and was removed for now rather than debugged immediately. The fix is known, just not done yet.
README for the repo	Not started	The GitHub repo is public but has no README yet explaining what it is or how to run it.

5. Suggested Learning Path

If you want to genuinely understand what's been built here — not just operate it, but follow the reasoning and eventually make changes yourself — here's a concrete order to learn things in:

Step 1 — Python basics, hands-on

The single best free resource for this is **Automate the Boring Stuff with Python** (the full book is free online at automatetheboringstuff.com). It's project-based, not academic - you'll be manipulating files and writing small real programs within the first few chapters. **Actually type the code yourself and run it** - don't just read it. Give this real time (a few weeks of regular practice, not a weekend).

Step 2 — Read this project's own code, in order of simplicity

Once basics feel comfortable, come back to this repository and read, in this order:

1. `src/resort_atv_voice/config.py` - just constants and settings, no logic. Good warm-up.
2. `src/resort_atv_voice/small_talk.py` - small, simple string matching.
3. `src/resort_atv_voice/can_telemetry.py` - simple data lookup logic.
4. `src/resort_atv_voice/local_qa.py` - the real core logic (routing, keyword matching, the local/cloud split). More complex, but by now you'll have the vocabulary for it.
5. `src/resort_atv_voice/cloud_chat.py` - the new Gemini integration, small and focused, a good next read once the above make sense.

Step 3 — Make one small, real change yourself

Not something done for you - something you write, reviewed afterward. A safe first one: add a new phrase to `GREETING_RESPONSE` or `THANKS` in `small_talk.py`, run the test suite (`pytest tests/`), watch it pass. Small and low-stakes, but it's the real loop - write code, test it, see it work - repeated, is how this actually sinks in.

Step 4 — Broader concepts, in parallel

Alongside the above, it's worth building intuition for: what an AI language model actually is and the concept of **hallucination** (an AI confidently stating something false) - this directly explains the single biggest design decision in this whole project; roughly how speech-to-text and text-to-speech work conceptually; what "prompt engineering" and "grammar-constrained generation" mean (explains the router's core safety mechanism); and basic vehicle electronics (CAN bus), which matters once real ATV hardware gets connected.

If you only do one thing: read `CLAUDE.md` in the project folder. It is a detailed, continuously-updated journal of every real decision, bug, and fix made throughout this project, written in plain language, current as of today's work (2026-08-10). It will teach you this specific system far better than any generic course, because it explains not just what was built but *why*, including the mistakes made and reverted along the way.